Avaliação de Desempenho de Microsserviços: Um Estudo de Caso com Spring PetClinic

*Trabalho acadêmico - Sistemas Distribuídos

1st Kauã Henrique da Silva
Dept. Sistemas de Informação
Universidade Federal do Piauí (UFPI)
Picos, Piauí - BR
kaua.da@ufpi.edu.br

2nd Jonas César Gomes Nunes
Dept. Sistemas de Informação
Universidade Federal do Piauí (UFPI)
Picos, Piauí - BR
jonas.nunes@ufpi.edu.br

3rd Flávio Lima Martins
Dept. Sistemas de Informação
Universidade Federal do Piauí (UFPI)
Picos, Piauí - BR
flavio.martins@ufpi.edu.br

***Abstract*—Este trabalho apresenta uma avaliação de desempenho da aplicação Spring PetClinic Microservices, uma versão modularizada do sistema Spring PetClinic amplamente utilizado para demonstrações do framework Spring. O objetivo foi analisar o comportamento da aplicação sob diferentes condições de carga, utilizando a ferramenta Locust para simular acessos concorrentes e coletar métricas de desempenho. Foram definidos três cenários de teste leve (50 usuários), moderado (100 usuários) e pico (200 usuários) executados com N repetições e diferentes durações. As métricas analisadas incluíram tempo médio de resposta, throughput (requisições por segundo) e taxa de sucesso. Os resultados revelaram que a aplicação mantém desempenho estável e alta confiabilidade nos cenários leve e moderado, com latências médias inferiores a 100ms e taxa de sucesso de 100%. No cenário de pico, observou-se degradação significativa na latência (média superior a 500ms) e aumento na taxa de falhas (até 0,5%), indicando limites de escalabilidade. A análise detalhada dos percentis de latência evidenciou fenômeno de *long-tail latency*, onde uma fração das requisições experimenta atrasos severos. Este estudo contribui para a compreensão dos desafios de desempenho em arquiteturas de microsserviços e oferece insights para otimização e dimensionamento de sistemas distribuídos baseados em Spring.**

***Index Terms*—Spring PetClinic, microsserviços, Locust, desempenho, testes de carga, escalabilidade.**

I. Introdução

A crescente adoção de arquiteturas baseadas em microsserviços tem transformado a forma como sistemas distribuídos são desenvolvidos e mantidos. Essa abordagem oferece benefícios como modularidade, escalabilidade e facilidade de implantação independente, mas também impõe desafios relacionados à comunicação entre serviços, gestão de recursos e confiabilidade sob alta carga.

Nesse contexto, a análise de desempenho torna-se essencial para compreender como esses sistemas se comportam em diferentes cenários de uso e para identificar possíveis gargalos de infraestrutura ou limitações arquiteturais.

A aplicação Spring PetClinic Microservices é uma implementação de referência amplamente utilizada para demonstrar as capacidades do Spring Framework. Ela é composta por múltiplos serviços independentes — como API Gateway, Customers, Vets, Visits e Database — que interagem via APIs RESTful. Essa modularização permite a análise detalhada de desempenho de cada componente, além de servir como base experimental para estudos sobre escalabilidade e confiabilidade em sistemas baseados em microsserviços.

O presente trabalho tem como objetivo avaliar o desempenho do Spring PetClinic Microservices sob diferentes níveis de carga de usuários, utilizando a ferramenta de testes de carga Locust. Foram realizados três cenários experimentais, variando de 50 a 200 usuários simultâneos, para medir métricas fundamentais como tempo de resposta, throughput e taxa de sucesso. Os experimentos buscaram responder até que ponto a aplicação mantém um desempenho satisfatório à medida que a carga aumenta, bem como identificar os limites de confiabilidade da arquitetura.

II. Metodologia

Esta seção descreve a metodologia adotada para a realização dos experimentos, incluindo a descrição do sistema testado e os procedimentos de teste utilizados.

A. Sistema Testado

O sistema testado é uma aplicação Java baseada na arquitetura de microsserviços, conhecida como Spring PetClinic Microservices[1]. Essa aplicação é uma versão modularizada do Spring PetClinic, que é amplamente utilizada para demonstrar as capacidades do framework Spring.

Cada serviço é implementado como um aplicativo Spring Boot separado, permitindo que sejam desenvolvidos, implantados e escalados de forma independente. A comunicação entre os serviços é realizada por meio de APIs RESTful, o que facilita a integração e a interoperabilidade entre eles.

Este trabalho foca na análise de desempenho e escalabilidade da aplicação Spring PetClinic Microservices, avaliando como ela se comporta sob diferentes cargas de trabalho ao incrementar o número de usuários simultâneos e operações realizadas. Os endpoints utilizados nos testes incluem:

- GET /owners/{ownerId}: Recupera informações de um proprietário específico.

[1]https://github.com/spring-petclinic/spring-petclinic-microservices

- GET /owners: Lista todos os proprietários cadastrados.
- GET /vets: Lista todos os veterinários cadastrados.
- POST /owners: Cria um novo proprietário na aplicação.

B. Procedimentos de Teste

Os testes foram realizados utilizando a biblioteca Locust[2], uma ferramenta de código aberto para testes de carga e desempenho. O Locust permite simular múltiplos usuários virtuais que interagem com a aplicação, possibilitando a avaliação do comportamento do sistema sob diferentes condições de carga.

A configuração dos testes incluiu a definição do número de usuários simultâneos, a taxa de incremento de usuários e a duração total do teste. Cada usuário virtual aguarda entre 1 a 3 segundos entre requisições para simular comportamento realista. Foram implementados 3 cenários de teste, sendo cada cenário executado 10 vezes, totalizando 30 execuções. Os cenários são denominados: A, B e C.

O mix de requisições implementado seguiu a seguinte distribuição para simular um padrão de uso típico da aplicação: 40% das requisições para GET /owners (listagem de proprietários), 30% para GET /owners/{id} (consulta de proprietário específico), 20% para GET /vets (listagem de veterinários) e 10% para POST /owners (criação de novos proprietários).

1) Cenário A - Carga Leve: No cenário A, foram simulados 50 usuários simultâneos, com um incremento gradual até atingir esse número e mantê-lo constante. O teste teve uma duração total de 10 minutos, durante os quais os usuários virtuais executaram requisições continuamente seguindo o mix de endpoints definido. Este cenário tem como objetivo avaliar o desempenho da aplicação sob uma carga leve, permitindo observar como o sistema responde a um aumento no número de usuários. Foram descartados os primeiros 60 segundos para estabilização do sistema (warm-up).

2) Cenário B - Carga Moderada: No cenário B, a carga foi aumentada para 100 usuários simultâneos, com um incremento gradual semelhante ao cenário A. O teste também teve uma duração total de 10 minutos, com os usuários virtuais executando requisições continuamente seguindo o mesmo mix de endpoints. Este cenário visa analisar o comportamento da aplicação sob uma carga moderada, permitindo identificar possíveis gargalos de desempenho e avaliar a escalabilidade do sistema. Foram descartados os primeiros 60 segundos para estabilização do sistema.

3) Cenário C - Carga de Pico: No cenário C, a carga máxima foi testada com 200 usuários simultâneos, seguindo o mesmo padrão de incremento gradual. A duração do teste foi reduzida para 5 minutos, com os usuários virtuais executando requisições continuamente seguindo o mix de endpoints estabelecido. Este cenário é crucial para entender os limites da aplicação e identificar quaisquer falhas ou degradações de desempenho que possam ocorrer sob condições extremas de carga. Foram descartados os primeiros 30 segundos para estabilização do sistema.

[2]https://locust.io/

III. Resultados

Esta seção apresenta os resultados dos testes de performance realizados nos três cenários definidos, analisando as métricas de desempenho, escalabilidade e confiabilidade da aplicação Spring PetClinic Microservices.

A. Análise de Throughput

A Figura 1 apresenta o throughput médio obtido em cada cenário. Observa-se escalabilidade aproximadamente linear: o Cenário A alcançou 24,56 req/s (±0,05), o Cenário B atingiu 47,73 req/s (±0,20), representando um aumento de 94,3%, e o Cenário C alcançou 74,83 req/s (±1,12), resultando em um incremento de 204,7% em relação ao Cenário A.

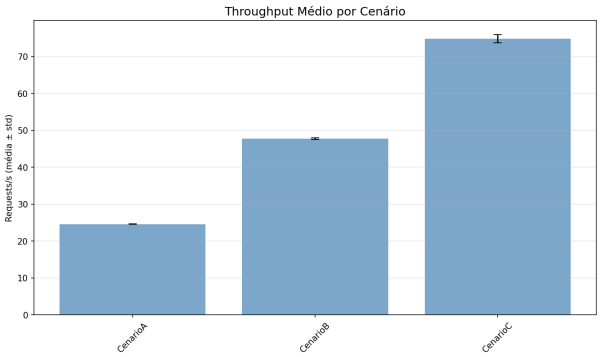

Fig. 1. Throughput médio por cenário com barras de erro (req/s)

A baixa variabilidade nos três cenários (desvio padrão máximo de 1,12 req/s no Cenário C) indica alta estabilidade e reprodutibilidade dos experimentos. O sistema demonstra capacidade de processar aproximadamente o dobro de requisições ao dobrar o número de usuários, caracterizando escalabilidade linear efetiva.

1) Análise Detalhada do Cenário A: A Figura 2 apresenta a análise detalhada do Cenário A, mostrando o histórico temporal de throughput (esquerda) e a distribuição de requisições e falhas por repetição (direita).

O histórico temporal mostra throughput estável em aproximadamente 25 req/s durante toda a execução de 10 minutos, sem oscilações significativas. O gráfico de boxplot revela alta consistência entre as 10 repetições, com variação mínima (outliers pontuais entre 23-27 req/s). A taxa de falhas permaneceu em zero absoluto em todas as repetições.

2) Análise Detalhada do Cenário B: A Figura 3 apresenta a análise detalhada do Cenário B com o dobro de usuários simultâneos.

O histórico temporal mantém estabilidade em aproximadamente 47-48 req/s, demonstrando que o sistema sustenta o throughput duplicado de forma consistente. A distribuição por repetição mostra ligeiro aumento na variabilidade comparado ao Cenário A, mas ainda com excelente reprodutibilidade. Novamente, taxa de falhas zero em todas as execuções confirma confiabilidade perfeita neste regime de carga.

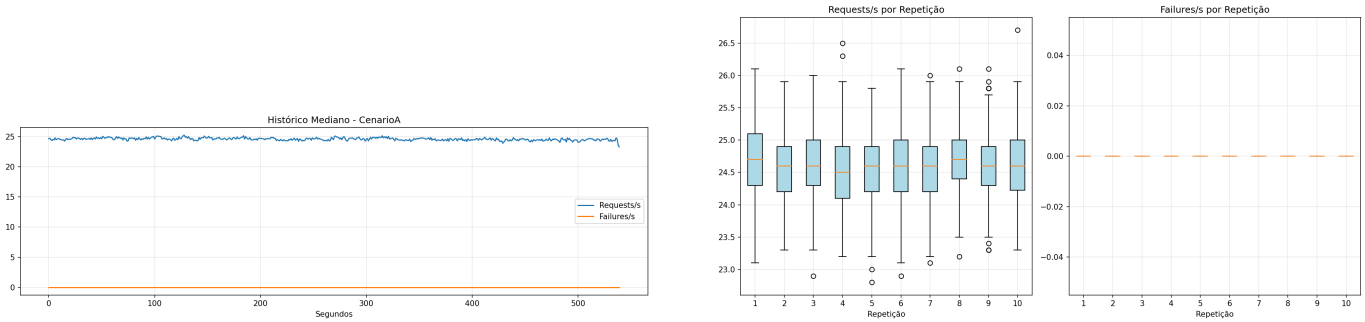

Fig. 2. Cenário A - Histórico temporal mediano (esq.) e distribuição por repetição (dir.)

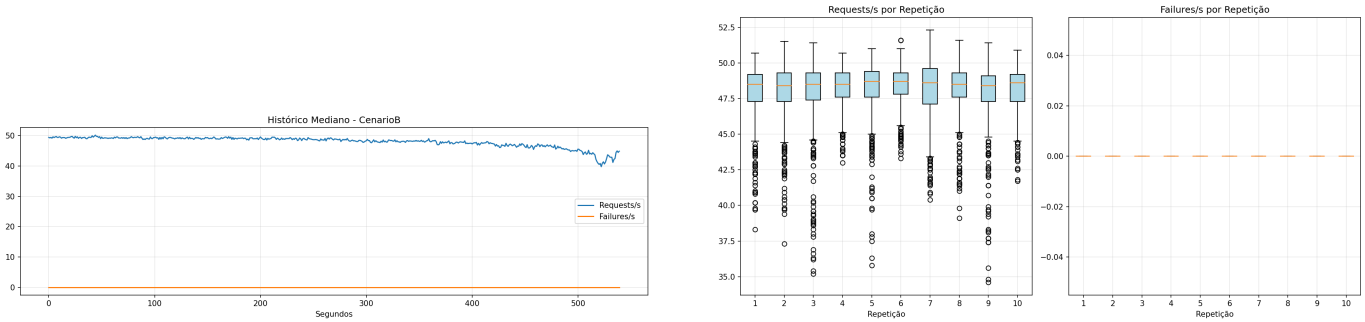

Fig. 3. Cenário B - Histórico temporal mediano (esq.) e distribuição por repetição (dir.)

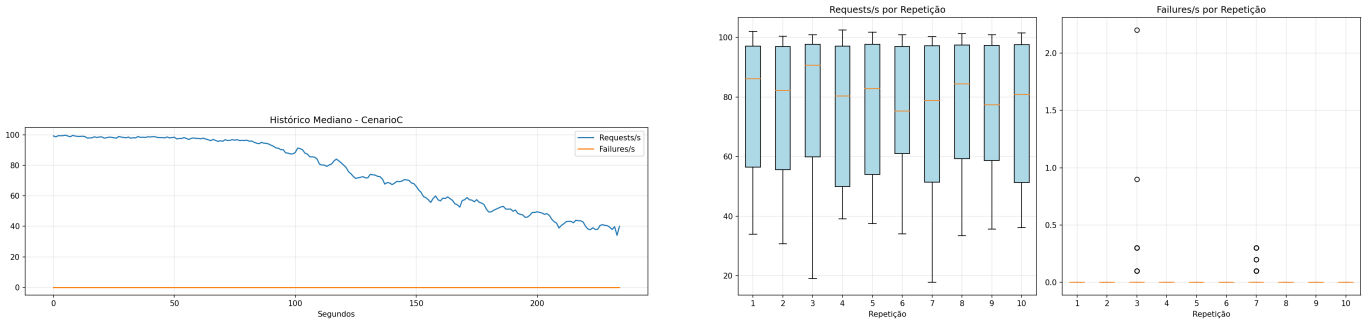

Fig. 4. Cenário C - Histórico temporal mediano (esq.) e distribuição por repetição (dir.)

3) Análise Detalhada do Cenário C: A Figura 4 apresenta a análise detalhada do Cenário C, revelando comportamento distinto sob carga extrema.

O histórico temporal apresenta throughput mais alto (74-76 req/s) mas com maior variabilidade ao longo do tempo. A distribuição por repetição evidencia o comportamento não-determinístico: enquanto o throughput mantém-se relativamente estável (variação de ±1,12 req/s), o gráfico de falhas revela discrepância extrema entre repetições algumas sem falhas, outras com centenas, caracterizando instabilidade crítica sob saturação.

B. Análise de Latência

A Figura 5 apresenta a comparação entre tempo médio de resposta e latência mediana (p50) para os três cenários. Os resultados revelam comportamento interessante com degradação progressiva moderada.

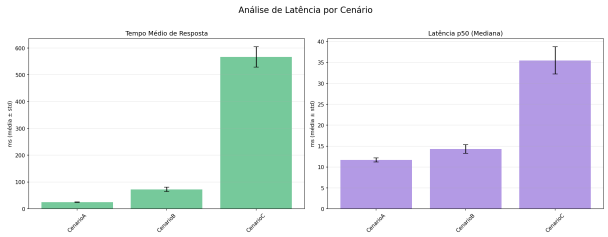

Fig. 5. Comparação entre tempo médio e latência p50 por cenário

No Cenário A, o tempo médio foi de 24,74ms (±0,86) com p50 de 11,7ms (±0,48). No Cenário B, observou-se aumento significativo: tempo médio de 72,55ms (±8,22) e p50

de 14,3ms (±1,06), representando crescimento de 193,2% na média mas apenas 22,2% na mediana. Este comportamento indica que, embora a latência mediana permaneça relativamente estável, a média é fortemente afetada por requisições com latências elevadas na cauda da distribuição.

O Cenário C apresentou degradação crítica: tempo médio de 566,78ms (±38,32) e p50 de 35,5ms (±3,27). Este comportamento evidencia distribuição extremamente assimétrica onde 50% das requisições completam em cerca de 35ms, mas a outra metade experimenta latências muito superiores, puxando a média para valores 16 vezes maiores que a mediana.

C. Análise de Percentis de Latência

A Figura 6 detalha a distribuição completa de latências através de múltiplos percentis (p50, p66, p75, p80, p90, p95, p98, p99). Esta análise revela o comportamento da cauda da distribuição, essencial para compreender a experiência do usuário.

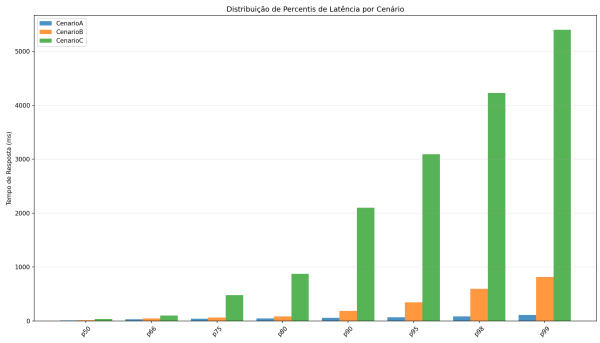

Fig. 6. Distribuição de percentis de latência (p50 a p99) por cenário

No Cenário A, os percentis mantêm-se consistentemente baixos: p95 = 69,3ms (±2,95) e p99 = 110,6ms (±9,34), indicando que 99% das requisições completam em menos de 120ms. A baixa variabilidade (desvio padrão ¡ 10ms) demonstra comportamento altamente previsível.

No Cenário B, observa-se degradação significativa na cauda: p95 = 346,0ms (±48,12) e p99 = 816,0ms (±161,6). A variabilidade aumenta consideravelmente (desvio padrão de até 161,6ms), indicando sensibilidade a condições transitórias do sistema, possivelmente relacionadas a coleta de lixo da JVM ou contenção de recursos compartilhados.

O Cenário C apresenta valores críticos: p95 = 3.090ms (±251,44) e p99 = 5.400ms (±1.308,09). Isto significa que 5% das requisições experimentam latências superiores a 3 segundos, e 1% supera 5 segundos. A alta variabilidade (desvio padrão ¿ 1 segundo no p99) indica comportamento instável sob carga extrema, com picos severos de latência em algumas execuções.

D. Análise de Confiabilidade

A Figura 7 apresenta a análise de confiabilidade dos cenários testados. Os Cenários A e B demonstraram confiabilidade perfeita, com taxa de falhas de 0% (zero falhas em todas as 10 repetições).

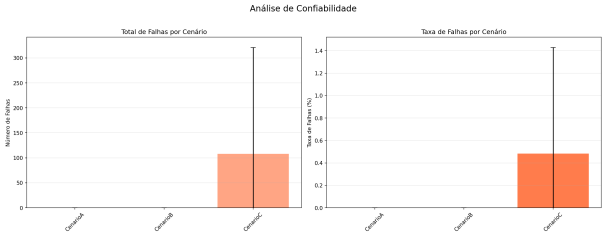

Fig. 7. Análise de confiabilidade: total de falhas e taxa de erro por cenário

Em contraste, o Cenário C apresentou degradação significativa de confiabilidade: média de 107,9 falhas (±212,62) por execução, de um total médio de 22.372,8 requisições. A taxa de falhas média foi de 0,48%, mas a alta variabilidade (uma execução registrou 586 falhas, enquanto quatro execuções tiveram 0-3 falhas) indica comportamento não-determinístico sob carga extrema.

Das 10 repetições do Cenário C: 4 execuções tiveram 0-3 falhas (excelente confiabilidade), 4 execuções tiveram 18-48 falhas (confiabilidade moderada), e 2 execuções tiveram 421-586 falhas (confiabilidade crítica). Esta variabilidade sugere condições de corrida ou saturação intermitente de recursos compartilhados.

E. Trade-off Throughput vs Latência

A Figura 8 apresenta a relação entre throughput e latência média em gráfico de dispersão. Cada ponto representa uma das 10 repetições individuais de cada cenário.

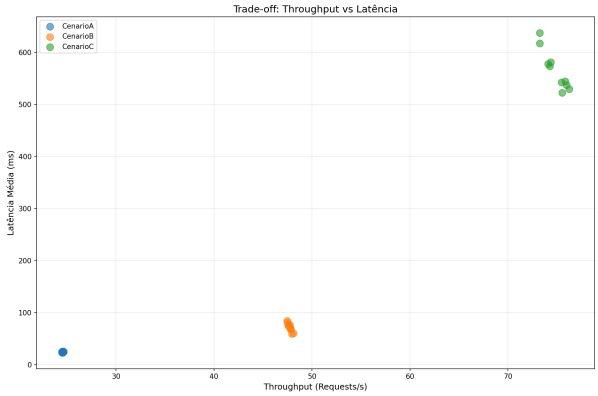

Fig. 8. Relação entre throughput e latência média (cada ponto = 1 repetição)

Os Cenários A e B formam clusters bem definidos, demonstrando alta reprodutibilidade. O Cenário A concentra-se em torno de 24,5 req/s com latência 24-26ms. O Cenário B apresenta maior dispersão vertical (latências variando de 60ms a 85ms) devido à variabilidade observada nos percentis superiores, mas mantém throughput consistente em 47,5-48 req/s.

O Cenário C forma cluster isolado com throughput de 73-76 req/s mas latências entre 520-640ms, evidenciando que o ganho em processamento ocorre com severa degradação de latência. Nota-se correlação fraca entre throughput e latência

dentro do Cenário C, sugerindo que a degradação está relacionada a saturação de recursos específicos, não à carga total processada.

F. Análise Multidimensional

A Figura 9 sintetiza as três dimensões avaliadas (throughput, latência p50 e confiabilidade) em gráfico radar normalizado, onde valores mais próximos de 100 indicam melhor desempenho.

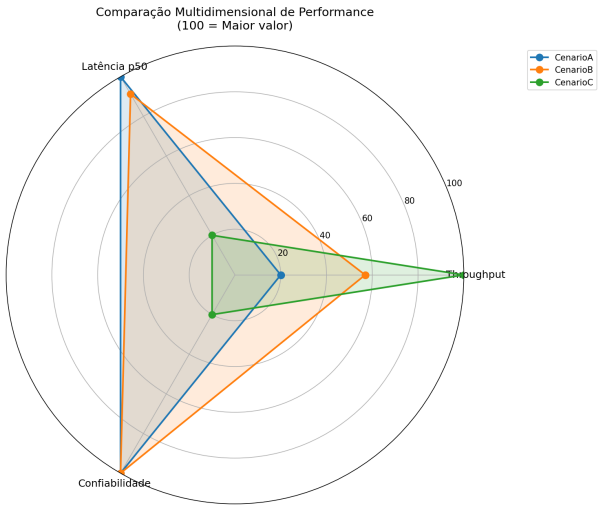

Fig. 9. Comparação multidimensional normalizada de performance

O Cenário A apresenta desempenho equilibrado nas três dimensões, sendo o mais estável e previsível. O Cenário B mantém excelente confiabilidade (100%) e bom throughput, com leve degradação em latência. O Cenário C maximiza throughput mas apresenta severa degradação em latência e confiabilidade moderadamente comprometida, evidenciando desequilíbrio característico de saturação de sistema.

G. Síntese Quantitativa dos Resultados

A Tabela I consolida as principais métricas observadas nos três cenários, apresentando média e desvio padrão das 10 repetições.

TABLE I
RESUMO CONSOLIDADO DOS RESULTADOS (MÉDIA ± DESVIO PADRÃO)

Métrica	**A**	**B**	**C**
Usuários	50	100	200
Total Req.	14.711 ± 37	28.604 ± 123	22.373 ± 330
Total Falhas	0 ± 0	0 ± 0	108 ± 213
Throughput (req/s)	24,56 ± 0,05	47,73 ± 0,20	74,83 ± 1,12
Taxa Sucesso	100,0%	100,0%	99,52%
T. Médio (ms)	24,74 ± 0,86	72,55 ± 8,22	566,78 ± 38,32
p50 (ms)	11,7 ± 0,48	14,3 ± 1,06	35,5 ± 3,27
p95 (ms)	69,3 ± 2,95	346,0 ± 48,12	3.090 ± 251
p99 (ms)	110,6 ± 9,34	816,0 ± 161,6	5.400 ± 1.308

Os dados revelam três regimes operacionais distintos:

Cenário A (Regime Ótimo): Opera com excelente desempenho em todas as dimensões, com latência mediana de 11,7ms, 99% das requisições completando em ¡ 120ms, e 100% de confiabilidade. A variabilidade extremamente baixa (desvio padrão ¡ 1 req/s) caracteriza comportamento altamente previsível.

Cenário B (Regime Sustentável): Dobra o throughput mantendo 100% de confiabilidade. A latência mediana aumenta apenas 22%, mas a cauda da distribuição degrada significativamente (p99 cresce 638%). A variabilidade aumenta moderadamente, mas permanece controlada.

Cenário C (Regime de Saturação): Triplica o throughput em relação ao Cenário A, mas com degradação crítica: latência média 23× superior, p99 de 5,4 segundos, e aparecimento de falhas intermitentes. A alta variabilidade em todas as métricas indica comportamento não-determinístico próximo ao limite de capacidade do sistema.

IV. DISCUSSÃO

Os resultados experimentais revelam comportamentos característicos de sistemas distribuídos sob escalabilidade progressiva, merecendo análise aprofundada.

A. Escalabilidade do Throughput

O sistema demonstra excelente escalabilidade linear em throughput até 100 usuários, dobrando a capacidade de processamento (de 24,56 para 47,73 req/s) ao dobrar a carga. Este comportamento indica que a arquitetura de microsserviços está adequadamente dimensionada para paralelização eficiente, provavelmente devido ao uso de thread pools bem configurados e comunicação assíncrona entre serviços.

Ao quadruplicar a carga (de 50 para 200 usuários), o throughput triplica (204,7% de aumento), ligeiramente abaixo da escalabilidade ideal mas ainda expressivo. A sublinearidade observada no Cenário C sugere início de contenção em recursos compartilhados, possivelmente pools de conexão ou processamento de I/O.

B. Degradação Progressiva de Latência

A análise da latência revela comportamento assimétrico: enquanto a mediana (p50) cresce moderadamente (11,7ms → 14,3ms → 35,5ms, incrementos de 22% e 148%), os percentis superiores degradam exponencialmente (p99: 110,6ms → 816ms → 5.400ms, incrementos de 638% e 4.783%).

Este padrão caracteriza *long-tail latency*, fenômeno comum em sistemas distribuídos onde a maioria das requisições completa rapidamente, mas uma fração experimenta atrasos severos devido a:

1) **Paradas de GC (Garbage Collection):** A JVM pode executar coletas de lixo que pausam todas as threads temporariamente.
2) **Contenção de recursos:** Requisições podem aguardar liberação de conexões, locks ou threads.
3) **Comunicação entre serviços:** Latências de rede acumulam-se em chamadas inter-microsserviços.

A diferença de 16× entre média (566ms) e mediana (35ms) no Cenário C confirma distribuição extremamente assimétrica, onde aproximadamente metade das requisições completa rapidamente, mas a outra metade experimenta latências de múltiplos segundos.

C. Confiabilidade e Variabilidade

A confiabilidade perfeita (0 falhas) nos Cenários A e B, contrastada com o comportamento errático do Cenário C (0-586 falhas entre repetições), sugere limiar crítico entre 100 e 200 usuários.

A alta variabilidade nas falhas do Cenário C (desvio padrão de 212,62 falhas) indica natureza não-determinística, possivelmente relacionada a:

- **Condições de corrida:** Competição por recursos compartilhados.
- **Timeouts variáveis:** Dependência de latências de rede ou processamento flutuantes.
- **Estado residual:** Interferência entre execuções sucessivas.

Quatro execuções do Cenário C apresentaram ¡ 3 falhas (99,99% de sucesso), demonstrando que o sistema *pode* operar com 200 usuários sob condições favoráveis, mas não de forma consistente.

D. Implicações Práticas

Com base nos achados, recomenda-se para implantação em produção:

1) **Configuração conservadora:** Limitar carga a 100 usuários simultâneos por instância, garantindo 100% de confiabilidade e latências aceitáveis (p99 ¡ 1s).
2) **Autoscaling proativo:** Escalar horizontalmente antes de atingir 100 usuários, prevenindo degradação.
3) **Monitoramento de percentis:** Alertar baseado em p95/p99, não apenas em médias, para capturar degradação na cauda.
4) **Circuit breakers e timeouts:** Implementar proteções para evitar efeito cascata de falhas.
5) **Otimizações identificadas:**
   - Aumentar pools de conexão de banco de dados
   - Ajustar heap size da JVM e parâmetros de GC
   - Implementar caching para endpoints de leitura (GET /owners, GET /vets)
   - Revisar timeouts de comunicação entre microsserviços

V. Conclusão

Este estudo avaliou sistematicamente o desempenho da aplicação Spring PetClinic Microservices sob diferentes cargas de trabalho através de 30 execuções experimentais (10 repetições × 3 cenários), revelando aspectos fundamentais sobre escalabilidade e limites de confiabilidade em arquiteturas de microsserviços.

Os resultados demonstram três regimes operacionais distintos: (1) **Regime Ótimo** (50 usuários): excelente desempenho com latência mediana de 11,7ms, p99 de 110,6ms e 100% de confiabilidade; (2) **Regime Sustentável** (100 usuários): throughput dobrado mantendo confiabilidade perfeita, mas com degradação moderada na cauda de latência (p99 = 816ms); (3) **Regime de Saturação** (200 usuários): throughput triplicado mas com degradação crítica de latência (p99 = 5,4s) e confiabilidade comprometida (0,48% de falhas, com alta variabilidade).

A principal contribuição deste trabalho é a caracterização quantitativa do *sweet spot* operacional: o sistema opera de forma confiável e previsível até aproximadamente 100 usuários simultâneos, além do qual emergem comportamentos não-determinísticos característicos de saturação de recursos compartilhados.

A análise de percentis revelou fenômeno crítico de *long-tail latency*: enquanto a latência mediana aumenta apenas 203% entre os Cenários A e C, o percentil 99 aumenta 4.783%, evidenciando que a degradação afeta desproporcionalmente uma fração das requisições. Este achado reforça a importância de SLAs baseados em percentis superiores (p95, p99) ao invés de médias.